

Project-Agricultural Chickens Production Pennsylvania Affected By COVID-19

Armin Mahdavi

2026-04-20

```
library(forecast)
```

```
## Registered S3 method overwritten by 'quantmod':  
##   method      from  
##   as.zoo.data.frame zoo
```

```
chicken <- read.csv("2018-2022 Chicken Production Data USDA.csv")
```

```
#observed that chicken$Value and chicken$Period are both chr columns, not numeric. Conversion to numeric
```

```
#Chicken$Value removes commas present.
```

```
chicken$Value <- as.numeric(gsub(",", "", chicken$Value))  
class(chicken$Value)
```

```
## [1] "numeric"
```

```
head(chicken$Value)
```

```
## [1] 40009000 51468000 48643000 42170000 49237000 45623000
```

```
any(is.na(chicken$Value))
```

```
## [1] FALSE
```

```
#Chicken$Period converts months to numeric by three-letter format, and the month-letters are all capital  
chicken$Period <- match(chicken$Period, toupper(month.abb))
```

```
#Sorting, which is important for time order.
```

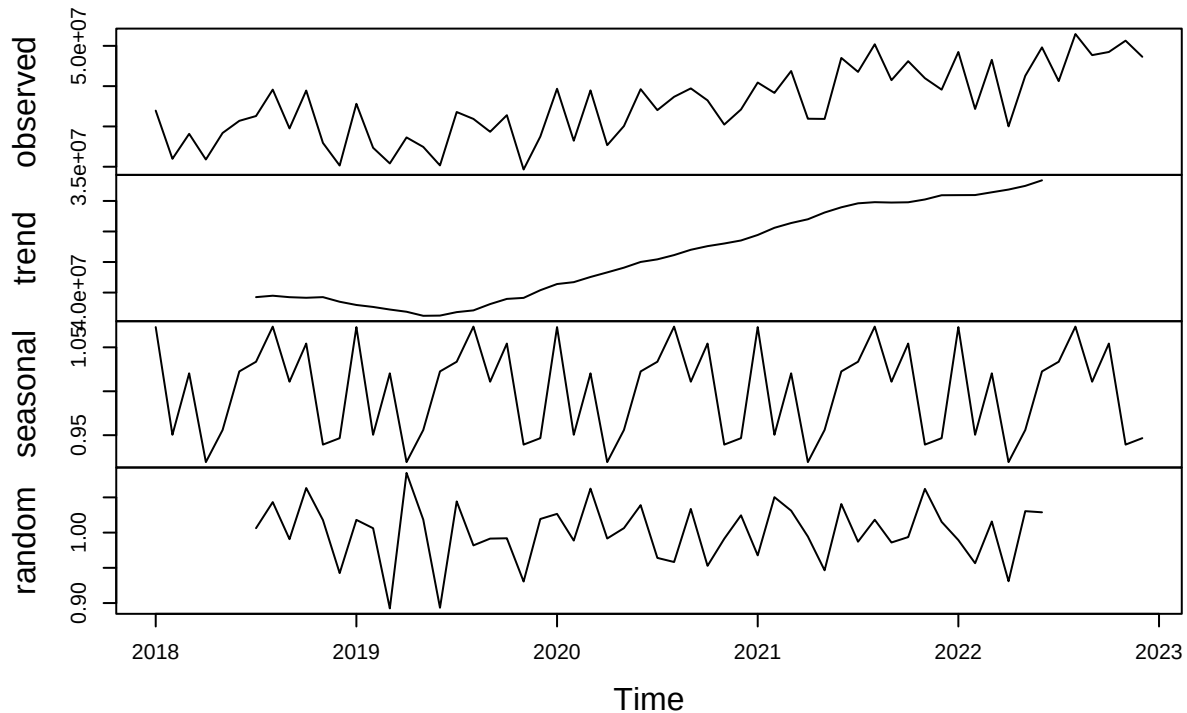
```
chicken <- chicken[order(chicken$Year, chicken$Period), ]
```

```
chicken_time <- ts(chicken$Value, start = c(min(chicken$Year), min(chicken$Period)), frequency = 12)  
str(chicken_time)
```

```
## Time-Series [1:60] from 2018 to 2023: 41962000 35982000 39083000 35896000 39190000 ...
```

```
#Seasonal Decomposition Multiplicative Model
decomposing_chickens<- decompose(chicken_time, type = "multiplicative")
plot(decomposing_chickens)
```

Decomposition of multiplicative time series



```
seasonal_pattern <- decomposing_chickens$figure
seasonal_pattern
```

```
## [1] 1.0730029 0.9504886 1.0203860 0.9193926 0.9559208 1.0225593 1.0335956
## [8] 1.0735540 1.0108901 1.0542736 0.9393133 0.9466229
```

```
decomposing_chickens$seasonal
```

```
##           Jan           Feb           Mar           Apr           May           Jun           Jul
## 2018 1.0730029 0.9504886 1.0203860 0.9193926 0.9559208 1.0225593 1.0335956
## 2019 1.0730029 0.9504886 1.0203860 0.9193926 0.9559208 1.0225593 1.0335956
## 2020 1.0730029 0.9504886 1.0203860 0.9193926 0.9559208 1.0225593 1.0335956
## 2021 1.0730029 0.9504886 1.0203860 0.9193926 0.9559208 1.0225593 1.0335956
## 2022 1.0730029 0.9504886 1.0203860 0.9193926 0.9559208 1.0225593 1.0335956
##           Aug           Sep           Oct           Nov           Dec
## 2018 1.0735540 1.0108901 1.0542736 0.9393133 0.9466229
## 2019 1.0735540 1.0108901 1.0542736 0.9393133 0.9466229
## 2020 1.0735540 1.0108901 1.0542736 0.9393133 0.9466229
## 2021 1.0735540 1.0108901 1.0542736 0.9393133 0.9466229
## 2022 1.0735540 1.0108901 1.0542736 0.9393133 0.9466229
```

```

t <- time(chicken_time)
trend_model <- lm(decomposing_chickens$trend ~ t, na.action = na.exclude)

future_t <- seq(max(t) + 1/12, by = 1/12, length.out = 12)
trend_forecast <- predict(trend_model, newdata = data.frame(t = future_t))
seasonal_forecast <- rep(seasonal_pattern, length.out = 12)

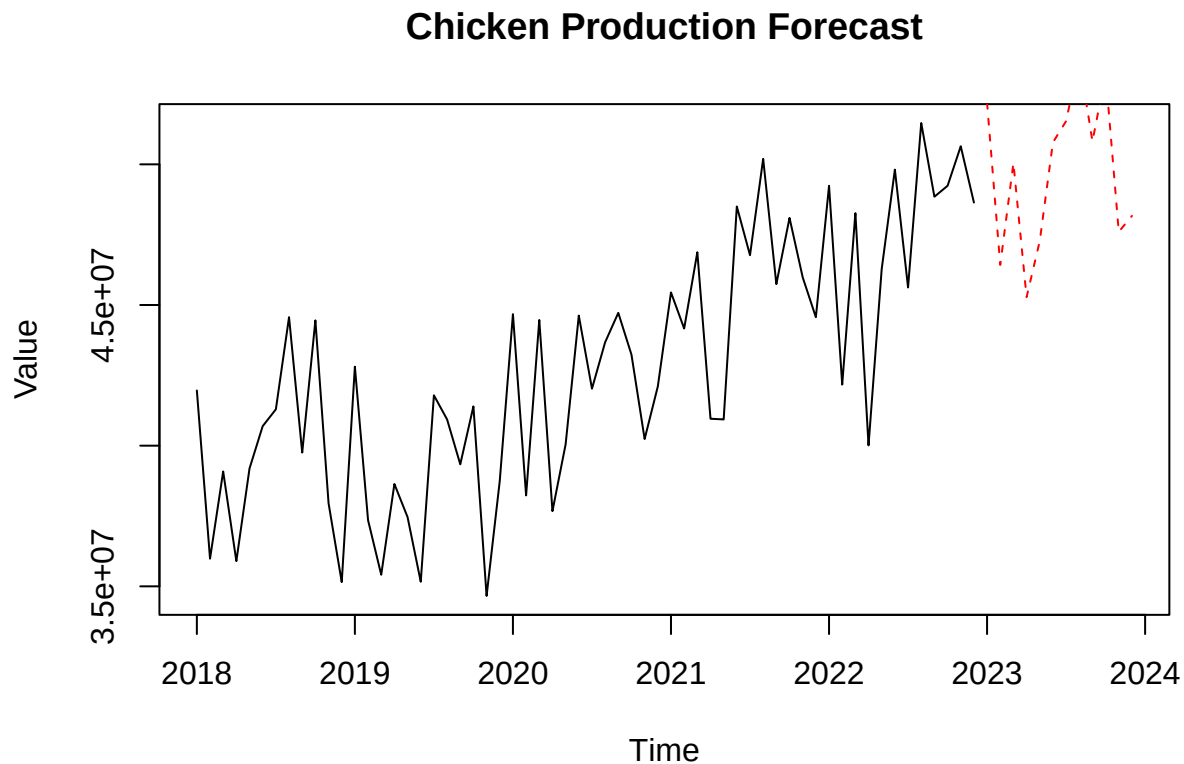
forecast_values <- trend_forecast * seasonal_forecast
forecast_values

##          1          2          3          4          5          6          7          8
## 52188389 46423489 50045569 45279842 47273866 50778004 51536911 53748325
##          9         10         11         12
## 50817243 53213206 47602356 48165912

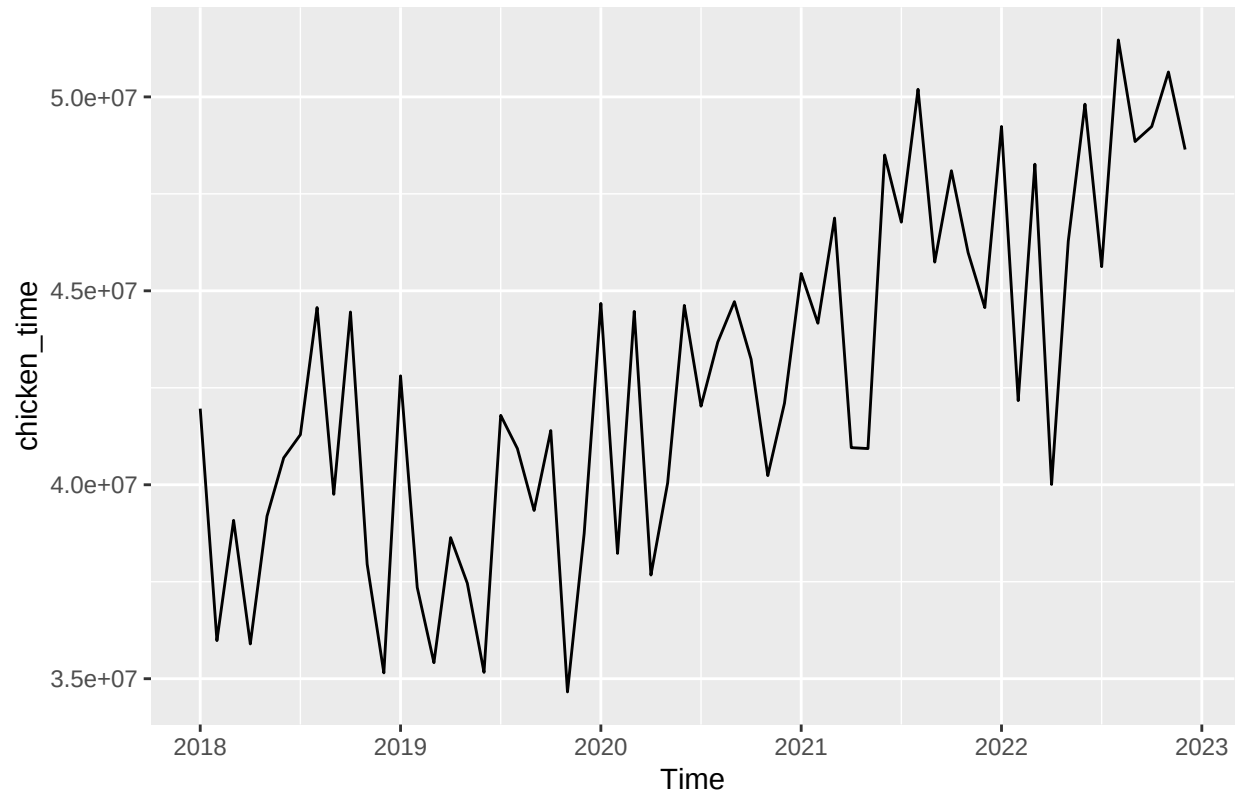
# last time point
end_time <- time(chicken_time)[length(chicken_time)]

# create forecast ts (12 months ahead)
forecast_ts <- ts(forecast_values,
                  start = end_time + 1/12,
                  frequency = 12)
plot(chicken_time, xlim = c(start(chicken_time)[1], end_time + 1),
     main = "Chicken Production Forecast", ylab = "Value");lines(forecast_ts, col = "red", lty = 2)

```

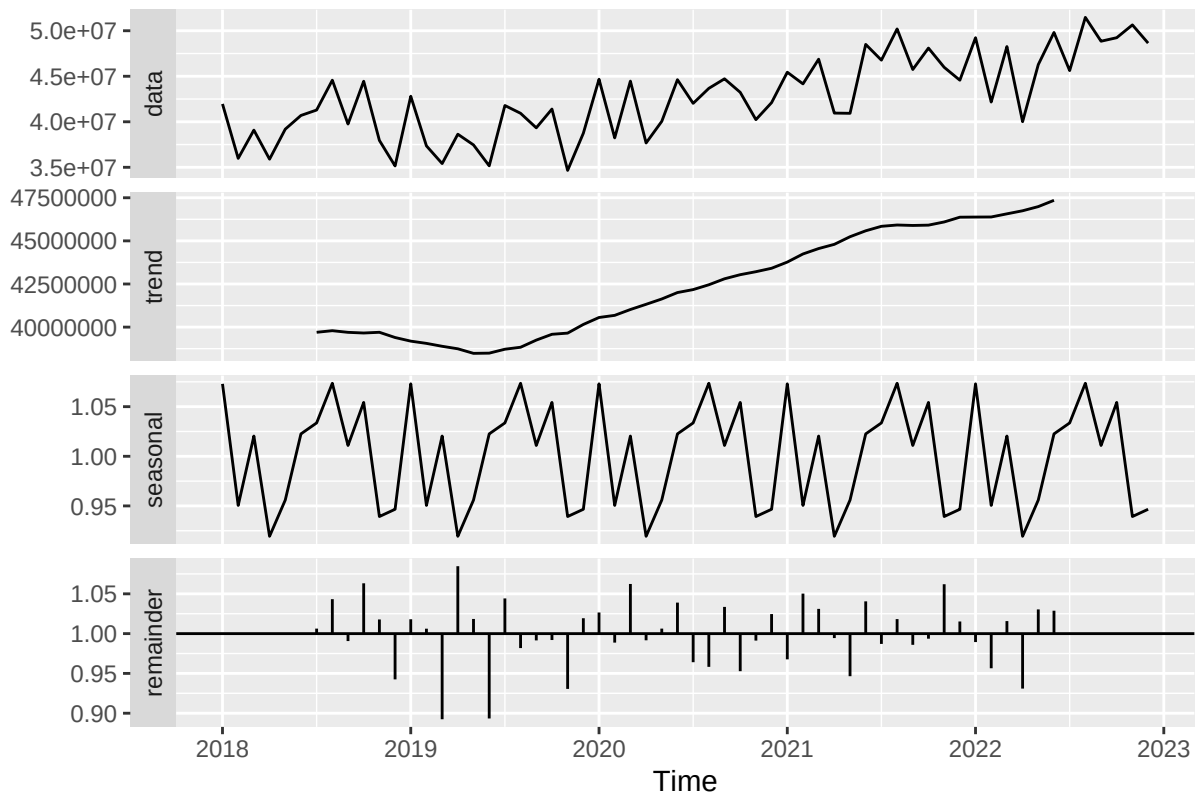


```
autoplot(chicken_time)
```



```
autoplot(decomposing_chickens)
```

Decomposition of multiplicative time series



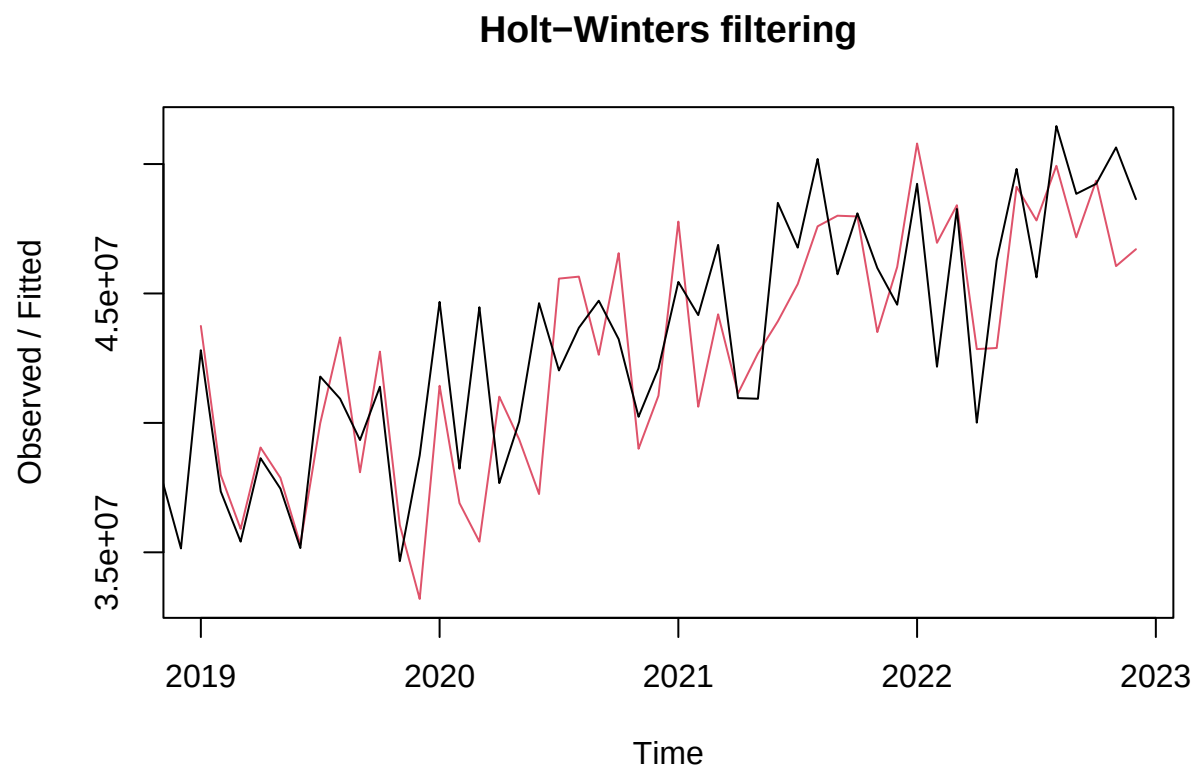
#Holt-Winters Multiplicative Model

```
hw_chickens <- HoltWinters(chicken_time, seasonal = "multiplicative")
hw_chickens
```

```
## Holt-Winters exponential smoothing with trend and multiplicative seasonal component.
##
## Call:
## HoltWinters(x = chicken_time, seasonal = "multiplicative")
##
## Smoothing parameters:
##   alpha: 0.2110384
##   beta : 0.0593853
##   gamma: 0.7322568
##
## Coefficients:
##           [,1]
## a  4.839613e+07
## b   2.372964e+05
## s1  1.077492e+00
## s2  9.554042e-01
## s3  1.066137e+00
## s4  9.066157e-01
## s5  9.985615e-01
## s6  1.081984e+00
## s7  1.010682e+00
## s8  1.111475e+00
```

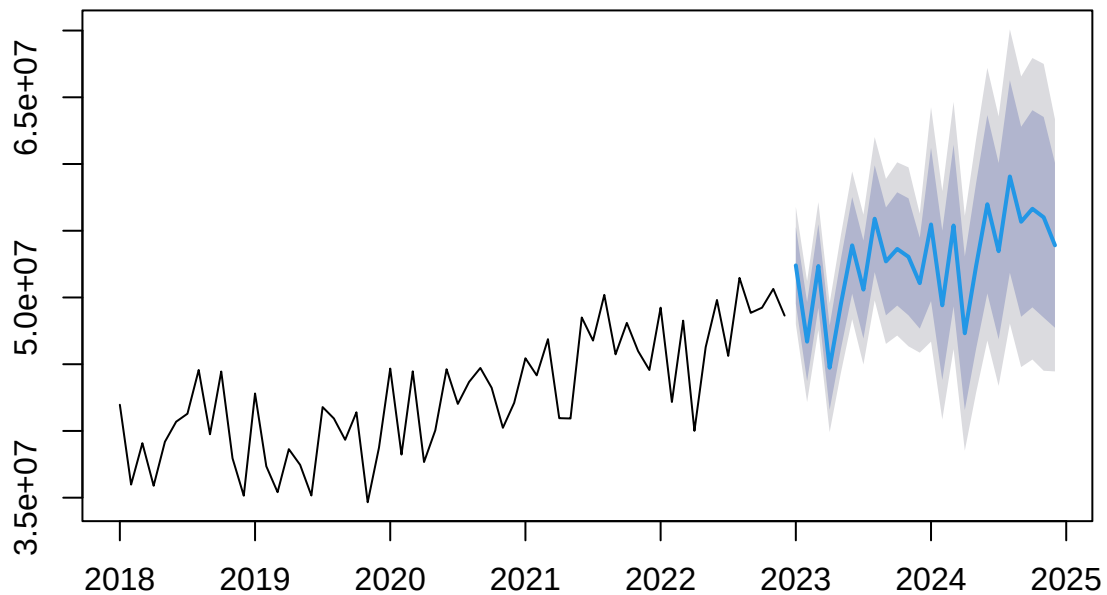
```
## s9  1.042972e+00  
## s10 1.056502e+00  
## s11 1.039961e+00  
## s12 9.966623e-01
```

```
plot(hw_chickens)
```



```
plot(forecast(hw_chickens))
```

Forecasts from HoltWinters

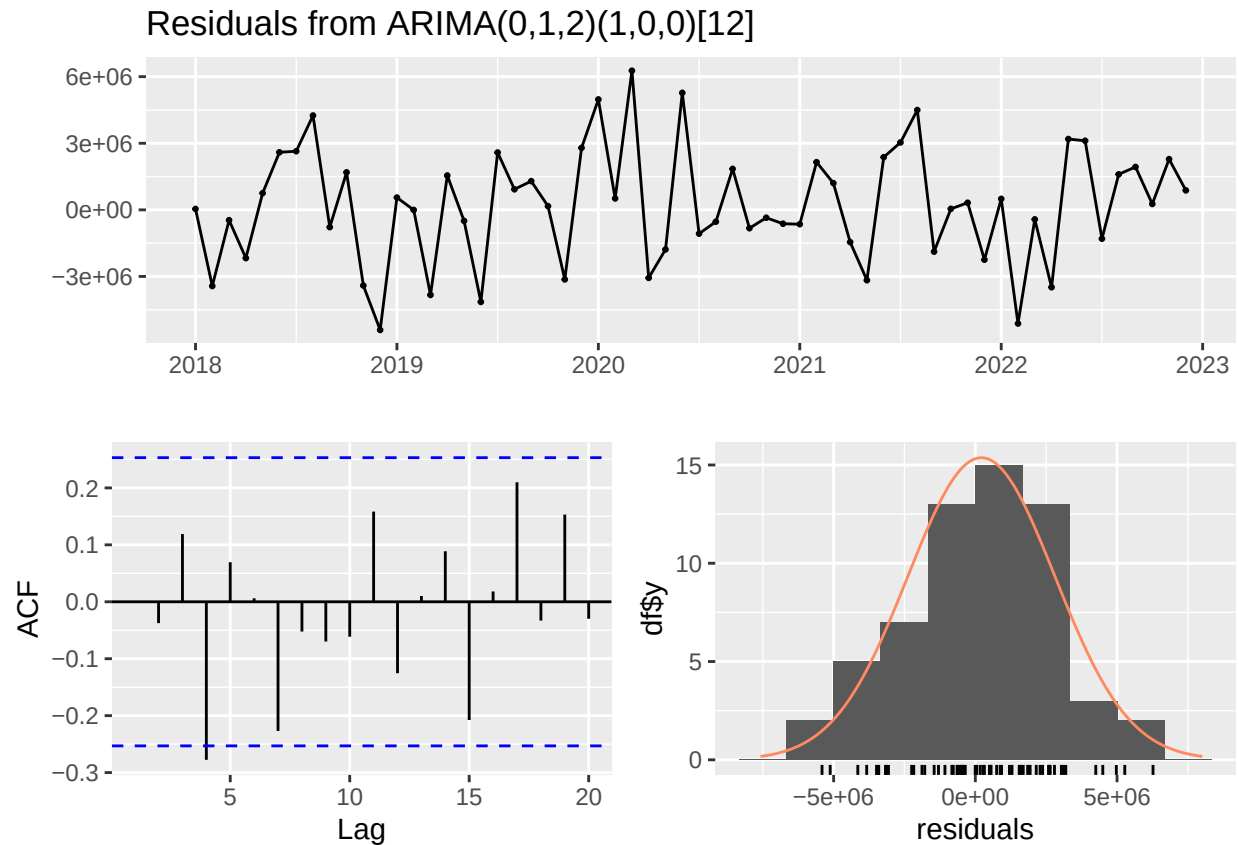


#ARIMA Mixed Model

```
arima_chickens<- auto.arima(chicken_time)
summary(arima_chickens)
```

```
## Series: chicken_time
## ARIMA(0,1,2)(1,0,0)[12]
##
## Coefficients:
##      ma1      ma2      sar1
##    -0.9275  0.3217  0.5945
## s.e.   0.1220  0.1665  0.1140
##
## sigma^2 = 7.161e+12:  log likelihood = -958.43
## AIC=1924.86  AICc=1925.6  BIC=1933.17
##
## Training set error measures:
##              ME      RMSE      MAE      MPE      MAPE      MASE      ACF1
## Training set 213901 2585176 2056380 0.1458906 4.919505 0.6712929 -0.001509238
```

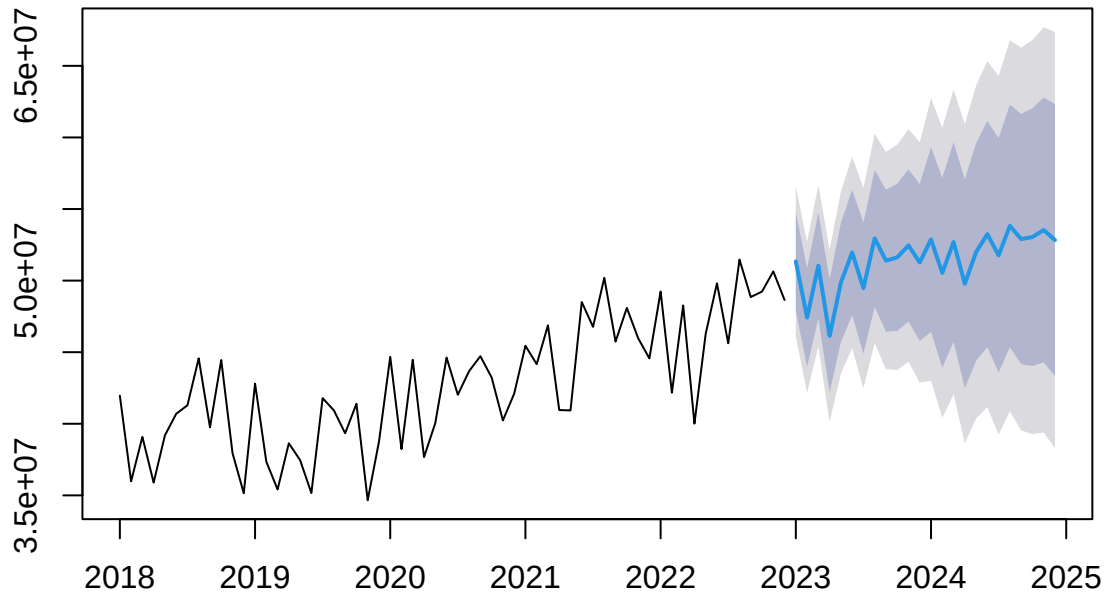
```
checkresiduals(arima_chickens)
```



```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(0,1,2)(1,0,0)[12]
## Q* = 14.027, df = 9, p-value = 0.1214
##
## Model df: 3.   Total lags used: 12
```

```
plot(forecast(arima_chickens))
```


Forecasts from ARIMA(0,1,2)(1,0,0)[12]



```
fc <- forecast(arima_chickens)
as.data.frame(fc)
```

##	Point	Forecast	Lo 80	Hi 80	Lo 95	Hi 95
##	Jan 2023	51340117	47910799	54769435	46095428	56584807
##	Feb 2023	47421397	43983084	50859710	42162951	52679843
##	Mar 2023	51044573	47350122	54739025	45394397	56694750
##	Apr 2023	46136789	42202840	50070738	40120334	52153245
##	May 2023	49865778	45706099	54025457	43504097	56227458
##	Jun 2023	51964189	47590413	56337964	45275076	58653301
##	Jul 2023	49474035	44896165	54051905	42472787	56475282
##	Aug 2023	52948599	48175354	57721844	45648550	60248647
##	Sep 2023	51391733	46430800	56352665	43804641	58978824
##	Oct 2023	51622379	46480607	56764152	43758716	59486042
##	Nov 2023	52457583	47141117	57774048	44326750	60588415
##	Dec 2023	51269276	45783678	56754874	42879777	59658774
##	Jan 2024	52872579	46423907	59321250	43010186	62734971
##	Feb 2024	50543093	43922400	57163786	40417617	60668569
##	Mar 2024	52696893	45734260	59659525	42048465	63345320
##	Apr 2024	49779457	42490909	57068005	38632585	60926330
##	May 2024	51996157	44395656	59596658	40372194	63620120
##	Jun 2024	53243558	45343413	61143703	41161329	65325787
##	Jul 2024	51763285	43574453	59952117	39239547	64287023
##	Aug 2024	53828742	45361059	62296424	40878538	66778945
##	Sep 2024	52903262	44165623	61640900	39540196	66266327
##	Oct 2024	53040370	44040869	62039870	39276821	66803918

```
## Nov 2024      53536857 44282901 62790812 39384153 67689560
## Dec 2024      52830467 43328869 62332065 38299027 67361907
```

```
autoplot(forecast(arima_chickens))
```

